

COLLEGE OF NATURAL AND COMPUTATIONAL SCIENCES

Department of Computer Science

Courses: Final Year Project I and Technical Report Writing in Computer Science

Final Year Project I Synopsis And Technical Report Writing in Computer Science Proposals

Group members

Name	ID
1. Yohannes Berhanu	UGR/9440/16
2. Hayredin Mohammed	UGR/2361/16
3. Bemnet Tesfaye	UGR/3266/16
4. Nahom Solomon	UGR/3155/16

Submission Date: October 6, 2026

Part 1: Final Year Project I Synopsis

Project 1: Multi-Class Coffee Leaf Disease Diagnosis

Project Title

Multi-Class Coffee Leaf Disease Diagnosis and Real-Time Offline Edge Inference System

Introduction

Coffee is a cornerstone of agricultural economies, but yields are frequently threatened by crop diseases such as Coffee Leaf Rust (*Hemileia vastatrix*), Cercospora Leaf Spot, Leaf Miner damage, and Phoma Leaf Spot. Early detection is critical for mitigation, yet traditional agricultural extensions rely on visual inspection by field experts, which is slow and inaccessible to rural farmers. This project presents an automated computer vision approach that provides real-time multi-class disease diagnosis. Designed specifically for low-cost mobile devices, the system performs fast, offline inference directly on the device without requiring cloud connectivity.

Statement of the Problem

Smallholder coffee farmers in remote regions lack immediate access to agronomists, leading to delayed disease identification and crop loss. Existing automated solutions often rely on cloud-based deep learning APIs, which fail in rural environments due to absent or unstable internet infrastructure. Furthermore, standard computer vision models are often computationally heavy and unsuitable for resource-constrained hardware. There is a need for a lightweight, offline computer vision pipeline that accurately classifies multiple coffee leaf diseases directly on edge devices.

Objectives

- **General Objective:**
  To develop a lightweight, offline deep learning system deployed on low-cost edge devices for real-time multi-class coffee leaf disease diagnosis.
- **Specific Objectives:**
  - To curate and preprocess a multi-class coffee leaf image dataset covering major pathogens (Coffee Leaf Rust, Cercospora, Leaf Miner, Phoma).
  - To train and optimize lightweight Convolutional Neural Network (CNN) architectures (e.g., MobileNetV3 or EfficientNet-Lite) using quantization and pruning techniques.
  - To deploy the optimized model using TensorFlow Lite / ONNX Runtime on a mobile smartphone application for offline field operation.
  - To evaluate model performance in terms of accuracy, latency, power consumption, and memory footprint on low-cost edge hardware.

Scope and Limitation

- **Scope:** The system focuses on identifying four primary coffee leaf conditions (Leaf Rust, Cercospora, Leaf Miner, Phoma) alongside healthy leaves using RGB imagery captured via smartphone cameras. It operates completely offline once deployed.
- **Limitation:** The diagnostic scope is restricted to coffee leaves and does not extend to soil analysis or stalk/cherry diseases. Diagnostic performance may vary under extreme lighting conditions or severely blurred camera inputs.

Significance of The Proposed Project

This system empowers rural coffee farmers with an accessible, immediate diagnostic tool that functions without internet dependency. By enabling early-stage detection, farmers can apply targeted treatments, reducing chemical overuse, preserving crop yields, and mitigating financial loss.

Project 2: Bidirectional Amharic<–>English Neural Machine Translation

Project Title

Bidirectional Amharic<->English Neural Machine Translation Using Custom Morphological Subword Segmentation and PEFT

Introduction

Natural Language Processing (NLP) for low-resource and morphologically complex languages presents unique challenges in translation accuracy. Amharic, the official working language of Ethiopia, utilizes the Ge'ez script and features a complex root-and-pattern morphological structure. Standard translation pipelines rely on byte-pair encoding (BPE) trained on English-centric corpora, which arbitrarily fragments Amharic verbs and nouns, degrading overall translation fidelity. This project presents a bidirectional Amharic<–>English Neural Machine Translation (NMT) system leveraging morphology-aware subword segmentation combined with Parameter-Efficient Fine-Tuning (PEFT/QLoRA) on multilingual pretrained models.

Statement of the Problem

Existing commercial and open-source translation models underperform on Amharic–English language pairs. Standard tokenization techniques treat morphologically complex Amharic root-and-pattern structures inefficiently, creating out-of-vocabulary (OOV) tokens and breaking semantic coherence. Additionally, fully fine-tuning massive sequence-to-sequence models requires high-end computational infrastructure that is often unavailable. A solution is needed that improves translation quality while remaining computationally manageable to train and fine-tune.

Objectives

- **General Objective:**
  To construct a high-quality, parameter-efficient bidirectional Amharic–English translation system using customized morphological tokenization.
- **Specific Objectives:**
  - To collect, clean, and align a parallel Amharic–English translation corpus from public and domain-specific sources.
  - To build a custom subword tokenizer tailored to Amharic morphological structures (root-and-pattern prefixing/suffixing).
  - To apply Quantized Low-Rank Adaptation (QLoRA) and PEFT to fine-tune a pretrained multilingual model (e.g., Meta's NLLB-200 or mBART-50).
  - To evaluate output translations against baseline systems using BLEU, CHRF, and human evaluation metrics.

Scope and Limitation

- **Scope:** The project is limited to text-to-text bidirectional translation between formal written Amharic and English across general and formal news/technical domains.
- **Limitation:** The model will not address non-standard dialectal variations, informal social media slang, or speech-to-speech translation pipelines.

Significance of The Proposed Project

By resolving tokenization bottlenecks for Amharic, this project advances natural language processing for Ge'ez-script languages. It provides better cross-lingual access to educational, legal, and administrative documents for local research and translation tools.

Project 3: Sign Language Translation System

Project Title

Continuous Sign Language Translation System Using Geometric Motion Keypoints and Sequence Modeling

Introduction

Communication barriers between the deaf and hard-of-hearing community and the hearing public remain significant. Automatic Sign Language Translation (SLT) aims to bridge this gap by converting continuous gesture streams into written text and audible speech. Traditional approaches process raw video frames through computationally heavy 3D Convolutional Neural Networks (3D-CNNs), requiring significant processing power and processing latency. This project implements a lightweight continuous SLT pipeline that extracts skeletal motion geometry using keypoint tracking instead of raw pixels, passing the resulting spatial-temporal vectors into sequence models.

Statement of the Problem

Raw video processing in sign language translation suffers from high computational complexity, vulnerability to background clutter, and sensitivity to variations in lighting and skin tones. 3D-CNN models are heavy, difficult to train, and challenging to run in real-time scenarios. An efficient system is needed that isolates body and hand motion representations from background noise while maintaining low latency and high accuracy in continuous sign translation.

Objectives

- **General Objective:**
  To design and build an efficient, continuous sign language translation system using skeletal keypoint geometric tracking and sequence-to-sequence neural architectures.
- **Specific Objectives:**
  - To extract real-time 3D hand (21 coordinates per hand) and pose keypoints using MediaPipe framework from video feeds.
  - To normalize coordinates against invariant frame scales and user distance variations.
  - To train a temporal sequence network (e.g., Transformer/LSTM or Spatial-Temporal Graph Neural Network) on structural keypoint sequences to predict target text.
  - To integrate a Text-to-Speech (TTS) module to generate real-time synthesized audio output.

Scope and Limitation

- **Scope:** The system covers continuous sequence gestures within a defined sign language vocabulary, focusing on real-time video stream processing to generate text and synthesized audio output.
- **Limitation:** Translation performance depends on full visibility of the upper body and hands. Occluded movements or extreme side-angle camera views may reduce keypoint detection accuracy.

Significance of The Proposed Project

Replacing heavy frame-based processing with keypoint coordinate geometry dramatically reduces computational load and eliminates background noise dependencies. This creates a scalable pathway toward practical, low-latency sign language translation tools on everyday devices.

Part 2: Technical Report Writing in Computer Science Proposals

Proposal 1: RAG vs. Fine-Tuning for LLMs

Title

RAG vs. Fine-Tuning: A Comparative Analysis for Domain-Specific Question Answering in Large Language Models

Introduction

Adapting general-purpose Large Language Models (LLMs) to specialized, domain-specific tasks is a core challenge in modern artificial intelligence. Two primary paradigms have emerged to address this: **Retrieval-Augmented Generation (RAG)**, which dynamically injects relevant context into the prompt at inference time, and **Domain Fine-Tuning (e.g., LoRA/QLoRA)**, which directly modifies the internal weights of the model through supervised learning on domain datasets. While both aim to improve accuracy and reduce errors in specialized domains (such as legal, medical, or proprietary enterprise data), they operate on fundamentally different technical principles, resource budgets, and operational workflows.

Problem Statement

Engineers and decision-makers lack clear empirical criteria when deciding whether to implement RAG, Fine-Tuning, or a hybrid approach for domain-specific Question Answering (QA). Fine-tuning can be computationally expensive, struggles with dynamic/frequently updating knowledge, and requires continuous retraining. Conversely, RAG incurs higher inference-time latency, depends heavily on embedding quality, and can suffer from context-window limits and retrieval failures. Without a structured comparative benchmark analyzing factual accuracy, hallucination rates, update agility, and operational costs, organizations risk selecting suboptimal architectures for enterprise AI deployments.

Objectives and Scope

The primary objective of this technical report is to conduct a systematic comparative analysis between Retrieval-Augmented Generation (RAG) and Parameter-Efficient Fine-Tuning (PEFT/LoRA) for domain-specific QA tasks.

Scope includes:

- Evaluating both paradigms on a standardized, domain-specific dataset (e.g., technical documentation, legal contracts, or medical literature).
- Benchmarking model performance using open-weight models (e.g., Llama 3 / Mistral variants).
- Comparing static knowledge adaptation (fine-tuning) against dynamic external knowledge retrieval (vector search + RAG).
- Excluding full pre-training from scratch due to prohibitive computational requirements.

Expected Outcomes and Evaluation

This report will deliver a concrete decision framework and empirical trade-off matrix.

Evaluation Metrics:

- **Accuracy & Factuality:** Evaluated using standard benchmarks ($ROUGE$, $BLEU$, $BERTScore$) and LLM-as-a-judge evaluation frameworks (e.g., Ragas) for context relevance, faithfulness, and answer correctness.
- **Hallucination Rate:** Quantifying fabricated information produced by both approaches under out-of-domain edge cases.
- **Knowledge Agility:** Measuring time and compute effort required to update the system with new information (updating a vector index vs. model retraining).
- **Cost & Latency:** Analyzing end-to-end inference latency ($p95/p99$) and token/compute cost per query.

Proposal 2: Vector vs. Traditional Databases

Title

Vector Databases vs. Traditional Relational & NoSQL Databases: Comparative Analysis for High-Dimensional Semantic Search

Introduction

Traditional relational databases (PostgreSQL, MySQL) and NoSQL stores (MongoDB) have historically powered web application data management using exact-match queries, B-Trees, and text indexing. However, the rise of artificial intelligence and unstructured data (text, images, audio) has popularized **Vector Databases** (e.g., Qdrant, Milvus, Pinecone), which store high-dimensional embeddings and perform Approximate Nearest Neighbor (ANN) vector searches using algorithms like HNSW or IVF-PQ. Recently, traditional databases have introduced vector extensions (such as pgvector for PostgreSQL), blurring the boundary between specialized vector search engines and legacy database management systems.

Problem Statement

Development teams face architectural ambiguity when integrating semantic search and vector similarity into existing software infrastructure. Dedicated vector databases promise sub-millisecond high-dimensional search and massive scalability, but introduce data synchronization overhead, additional operational complexity, and fragmented persistence layers. Conversely, using vector extensions on traditional databases maintains ACID compliance and operational simplicity, but may suffer severe performance bottlenecks under high-concurrency or high-dimensional search loads. A rigorous comparative study is required to identify the performance thresholds where transitioning from a traditional database extension to a dedicated vector store becomes necessary.

Objectives and Scope

This technical report aims to benchmark specialized vector database architectures against traditional database engines enhanced with vector extensions for semantic search workloads.

Scope includes:

- Comparing dedicated vector databases (e.g., Qdrant / Milvus) against traditional databases with vector capability (e.g., PostgreSQL with pgvector).
- Evaluating performance across 768-dimensional and 1536-dimensional dense embedding vectors (standard OpenAI / HuggingFace outputs).
- Stress-testing read/write throughput during continuous vector ingestion and concurrent search queries.
- Analyzing operational complexity, transactional consistency, and data persistence guarantees.

Expected Outcomes and Evaluation

The report will yield quantitative performance benchmarks and an architectural guide for database selection in semantic search applications.

Evaluation Metrics:

- **Query Performance:** Measuring Queries Per Second (QPS) and search latency ($p95$/ $p99$ percentiles) under varying dataset scale (100k, 1M, and 10M vector records).
- **Retrieval Quality:** Evaluating $Recall@K$ and precision across vector indexing methods (HNSW, Flat, IVF).
- **Resource Efficiency:** Tracking memory (RAM) usage, CPU utilization, and disk storage overhead per million vectors.
- **Operational Ergonomics:** Assessing schema flexibility, ACID compliance, query language complexity, and backup/restore workflows.

Proposal 3: Cross-Platform Mobile Frameworks (2026)

Title

Client-Side Cross-Platform Mobile Frameworks: A Comparative Architectural and Performance Evaluation of React Native, Flutter, and Kotlin Multiplatform (KMP)

Introduction

Building mobile applications across iOS and Android natively requires maintaining two independent codebases in Swift and Kotlin, driving up development and maintenance costs. Cross-platform engineering solutions address this by allowing developers to share code across platforms. Three dominant paradigms represent modern cross-platform development: **React Native** (utilizing a JavaScript engine with native UI primitives via the New Architecture/Fabric), **Flutter** (using Dart and rendering directly to a canvas via custom engines like Impeller), and **Kotlin Multiplatform (KMP)** (sharing pure native Kotlin business logic while compiling directly to native binaries and rendering platform-native UI).

Problem Statement

Selecting a cross-platform framework is a high-stakes architectural decision with long-term implications for team velocity, UI responsiveness, binary size, and native hardware integration. Many performance comparisons rely on simple "hello world" benchmarks that fail to reflect real-world application complexity. Developers lack an up-to-date, empirical evaluation comparing how React Native's modernized architecture, Flutter's updated rendering engine, and KMP's native compilation trade off execution speed, UI frame consistency, shared code percentage, and developer experience under complex workload conditions.

Objectives and Scope

This technical report will conduct a comprehensive comparative analysis of React Native, Flutter, and Kotlin Multiplatform for modern cross-platform mobile application development.

Scope includes:

- Building identical benchmark mobile applications across all three frameworks containing real-world requirements: heavy list rendering, complex UI animations, local SQLite storage access, REST/WebSocket networking, and native device API access (camera/sensors).
- Analyzing performance on identical target hardware (iOS and Android devices).
- Evaluating architectural differences: JS bridge/JSI vs. direct canvas rendering vs. compiled native binaries.

Expected Outcomes and Evaluation

The report will provide an empirical performance scorecard and framework selection guide for engineering teams.

Evaluation Metrics:

- **Runtime Performance:** Measuring app launch time (cold/warm startup), Memory (RAM) footprint, CPU overhead, and Frame Rate consistency (FPS drops / jank during list scrolling and UI animations).
- **App Footprint:** Comparing compiled release binary sizes (.apk / .ipa) across platforms.
- **Code Reusability & Architecture:** Quantifying the percentage of shared business logic vs. platform-specific boilerplate code.
- **Developer Velocity & Ecosystem:** Scoring build times, hot-reload efficiency, community library availability, and native platform interop complexity.